

Finding Duplicate Files in Windows Using Python

Duplicate files have their uses, but when they are duplicated multiple times or under different names and in different directories, they can be a nuisance. This article shows readers how to use Python to eliminate such files in a Windows system.



Computer users often have problems with duplicate files. Sometimes we mistakenly create the same file again and again with different names, or copy one file to different locations with different names. So it becomes very difficult to find the duplicate file due to its different name. There is also the case of files with the same name having different content. In order to solve this problem, let's check out a new Python program.

Before jumping to the source code, I want to explain the principle of the code, which is based upon the integrity of the file. If two files have the same content, with the same or different names, then their MD5 hash (or other hash algorithm) must be the same. In this article, I am going to use the MD5 hash to find the integrity of files. In the first step, let's create and save the MD5 hash of all the files of all drives. See the basic code flow in Figure 1, which signifies the functionality of generating a database file that contains hashes of all files.

The function `create()` will be called by the user with arguments. It accesses all hard disk drives through the `get_drive()` function; then, it creates threads for each drive and calls the `search1()` function. The `search1()` function uses the `md5()` function to generate the MD5 hash for each file. In this way, the `search1()` function creates a Python default dictionary, which contains hashes as keys and files with paths as values. Finally, the `create()` function dumps the Python

default dictionary into a pickle file.
Let's discuss the code.

```
#program created by mohit
#official website L4wisdom.com
# email-id mohitraj.cs@gmail.com
```

The following modules will be used in the program. Do not worry about creating MD5 hashes because there is a module `hashlib` which will do this for you.

```
import os
import re
import sys
from threading import Thread
from datetime import datetime
import subprocess
import cPickle
import argparse
import hashlib
import collections
```

```
· Create a Python default dictionary

dict1 = collections.defaultdict(list)
```